Chair of Network Architectures and Services
School of Computation, Information, and Technology
Technical University of Munich

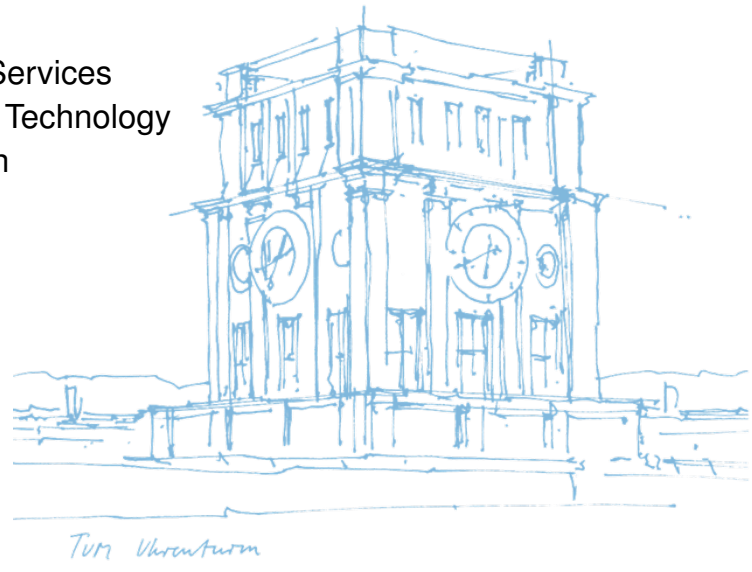
QUIC Improvements Using XDP

Intermediate talk for the Master's Thesis by

Dominik Ritzenhoff
advised by Johannes Späth, Marcel Kempf

Wednesday 23rd April, 2025

Chair of Network Architectures and Services
School of Computation, Information, and Technology
Technical University of Munich



Motivation

```
        - Remove rfc references.
      - consider using footnotes.
- consider using a figure to put there.
                    -
```

- HTTP/3 [1] commands 20% of internet traffic worldwide [2]
- Uses QUIC [3] as its L4 protocol
- QUIC throughput is not able to match TCP with increasing bandwidth [4]
- Packet I/O is QUIC's largest bottleneck in terms of its maximum achievable throughput [5]

XDP offers a remedy

XDP Primer

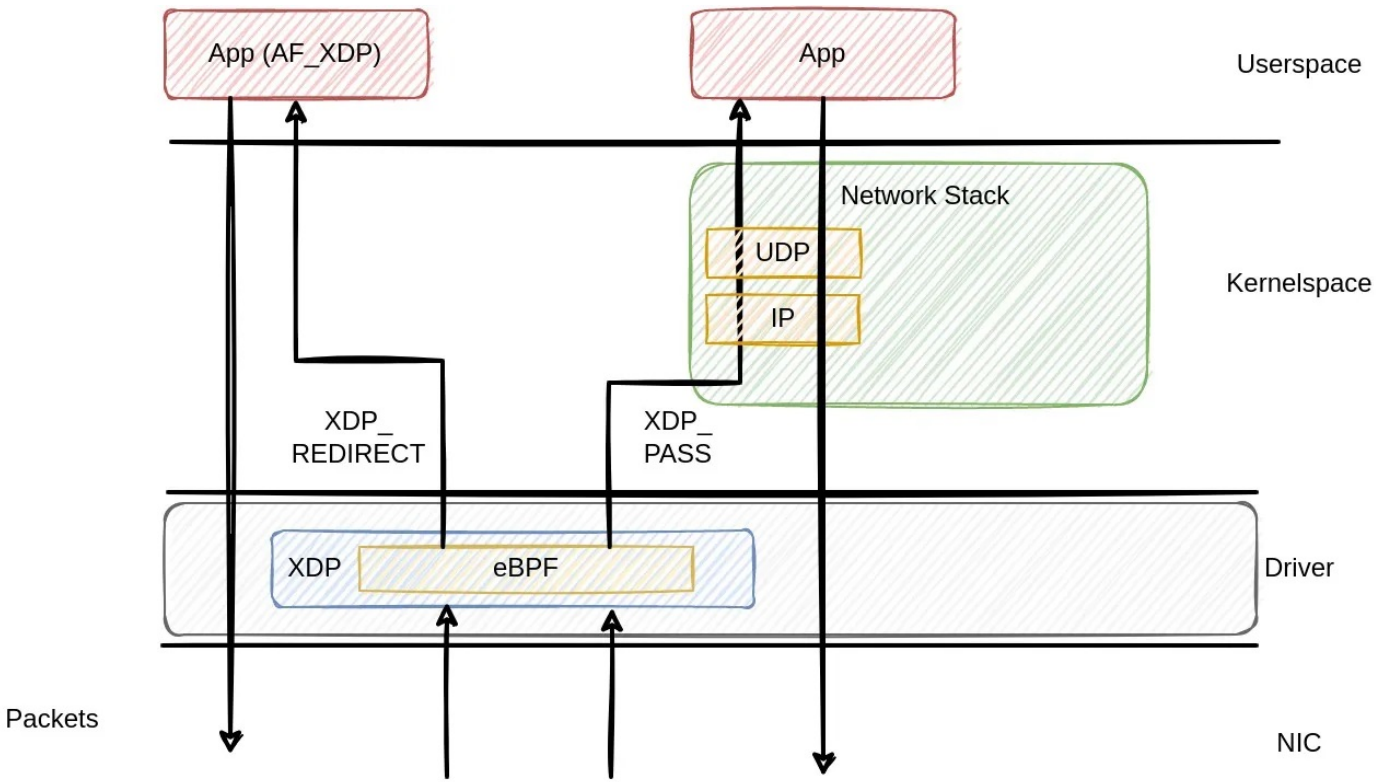
- XDP is a native Linux hook point for eBPF bytecode to run
- One of the earliest points the OS can access a packet
- Decide on an action for each packet
  - XDP_DROP
  - XDP_PASS
  - XDP_REDIRECT
  - XDP_ABORTED
  - XDP_TX

```
- use monospace font for these
- show where XDP lies in the network
 stack and what it replaces. figure
```

```
- Go into optimizations sendmmsg() + GRO
```

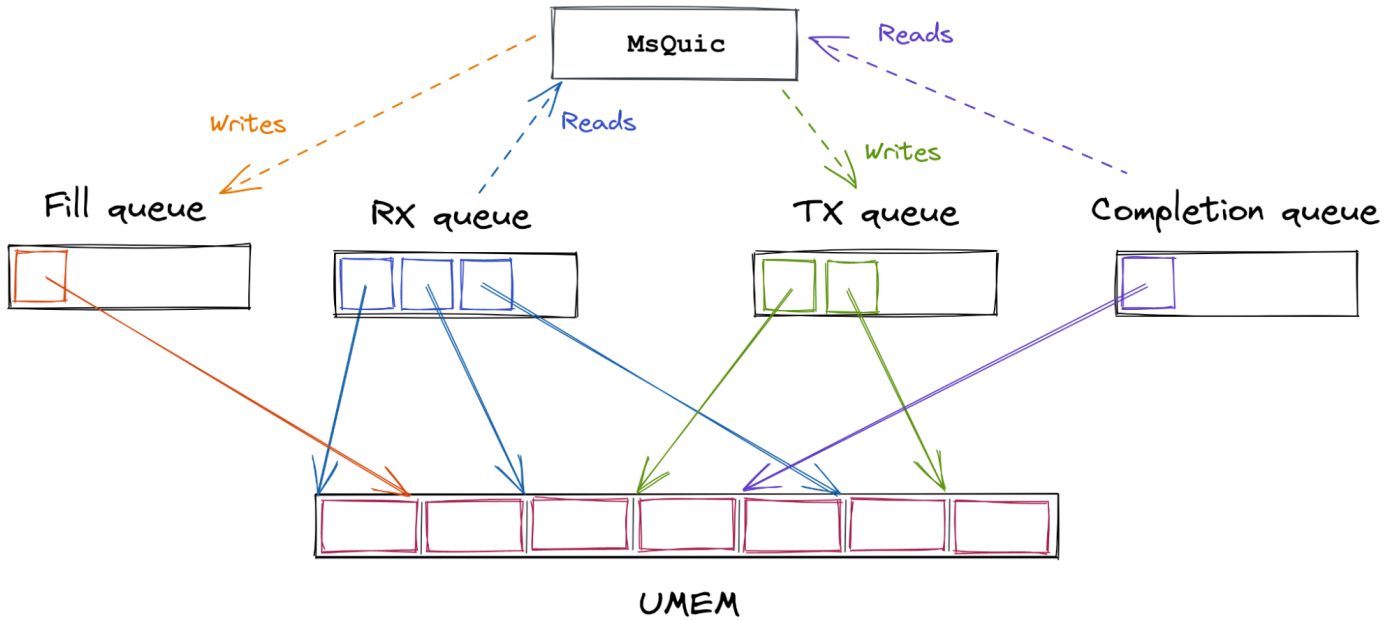
XDP Primer

- AF_XDP is an address family in Linux similar to AF_INET or AF_INET6
- It works with XDP and allows a packet skip the network stack
- Helps avoid costly per packet processing



AF_XDP Primer

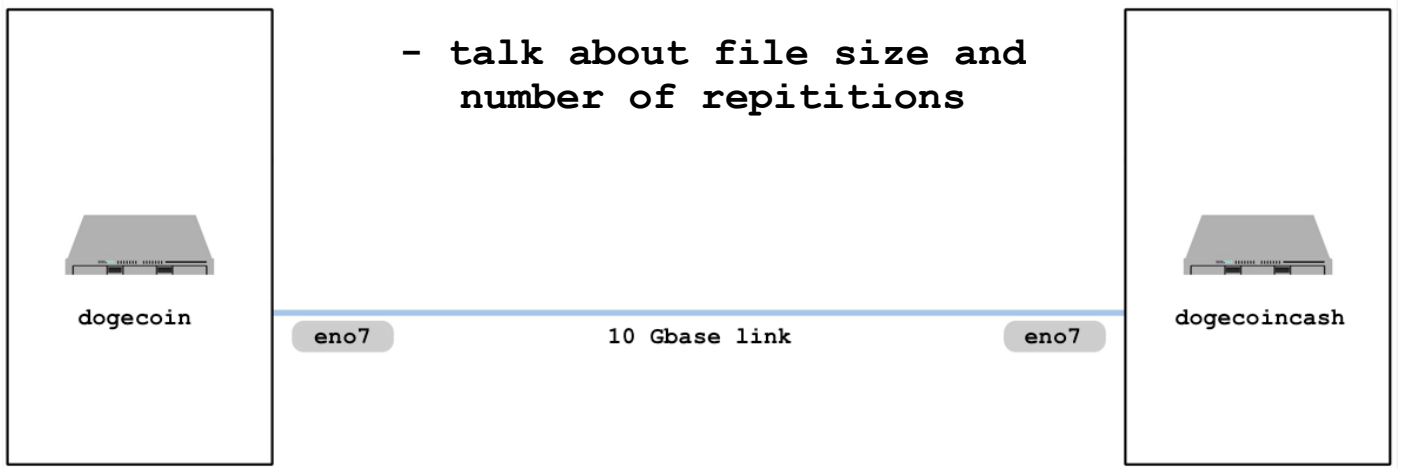
- AF_XDP uses UMEM with tx, rx, completion, and fill rings
- Each ring is single producer, single consumer
- Avoids locking contention **- spend time explaining this**
- Need a single system call to trigger multi-packet sends



Approach

- drop interop runner
- can add footnote for msquic but drop the citation

- Patch MsQuic [8] interop runner to support AF_XDP
- Run throughput tests in a client-server topology
- Generate Flame Graphs [9] to identify bottlenecks

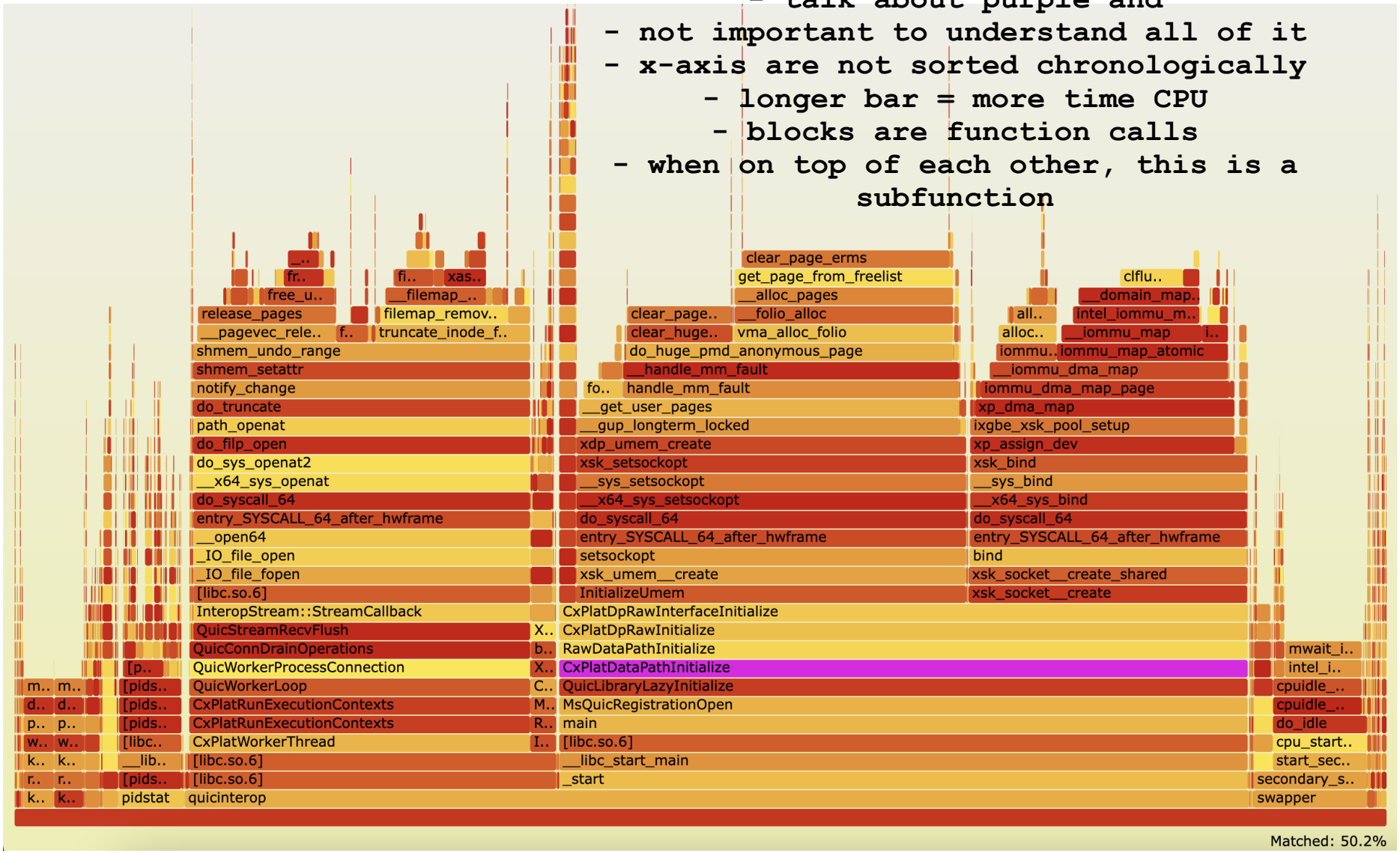


Optimization

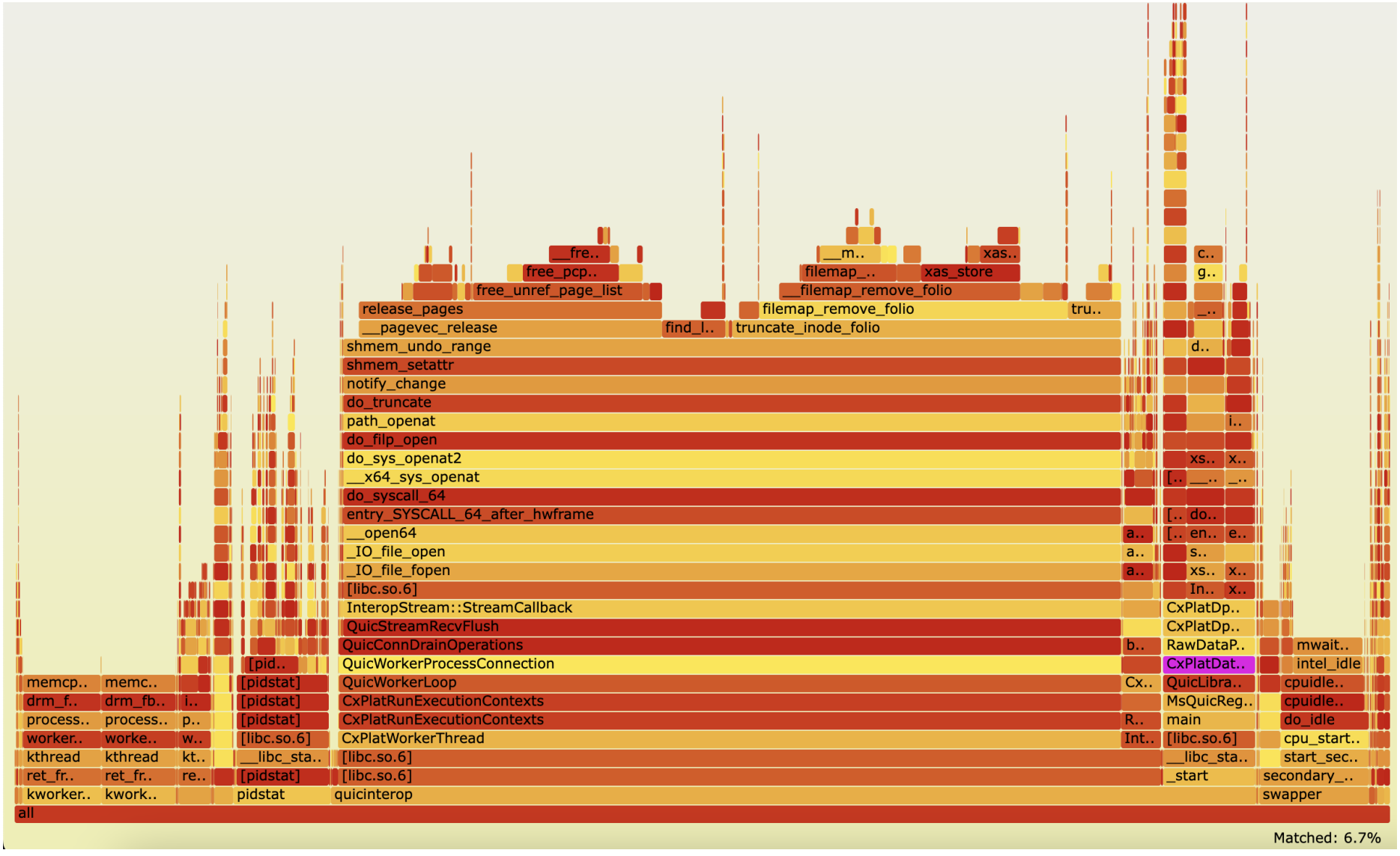
- be more specific about capture time.

Datapath long initialization time (3s perf trace) optimized by creating UMEM for only one queue on one interface

- talk about purple and
- not important to understand all of it
- x-axis are not sorted chronologically
- longer bar = more time CPU
- blocks are function calls
- when on top of each other, this is a subfunction

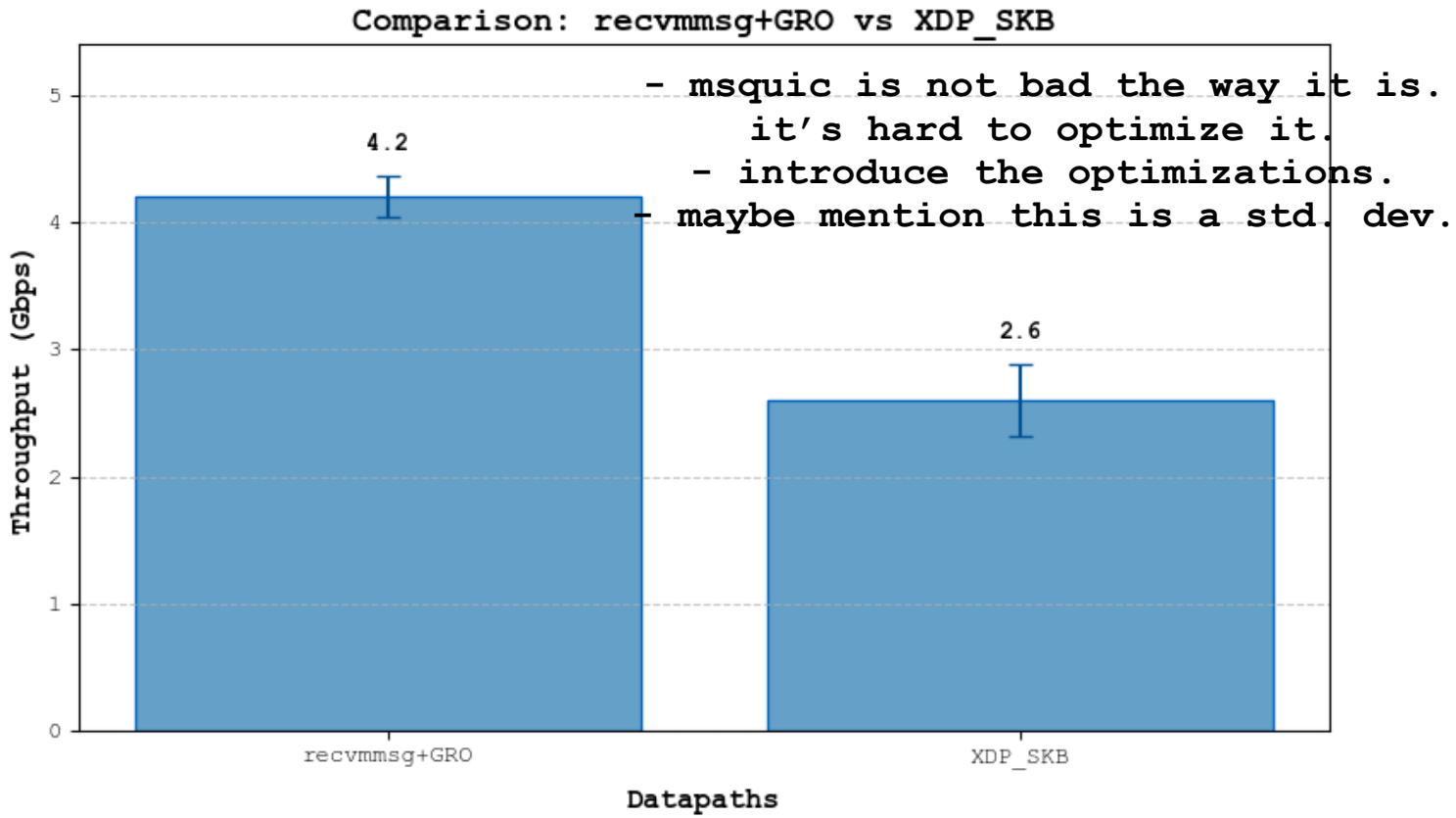


Optimization



Measurements

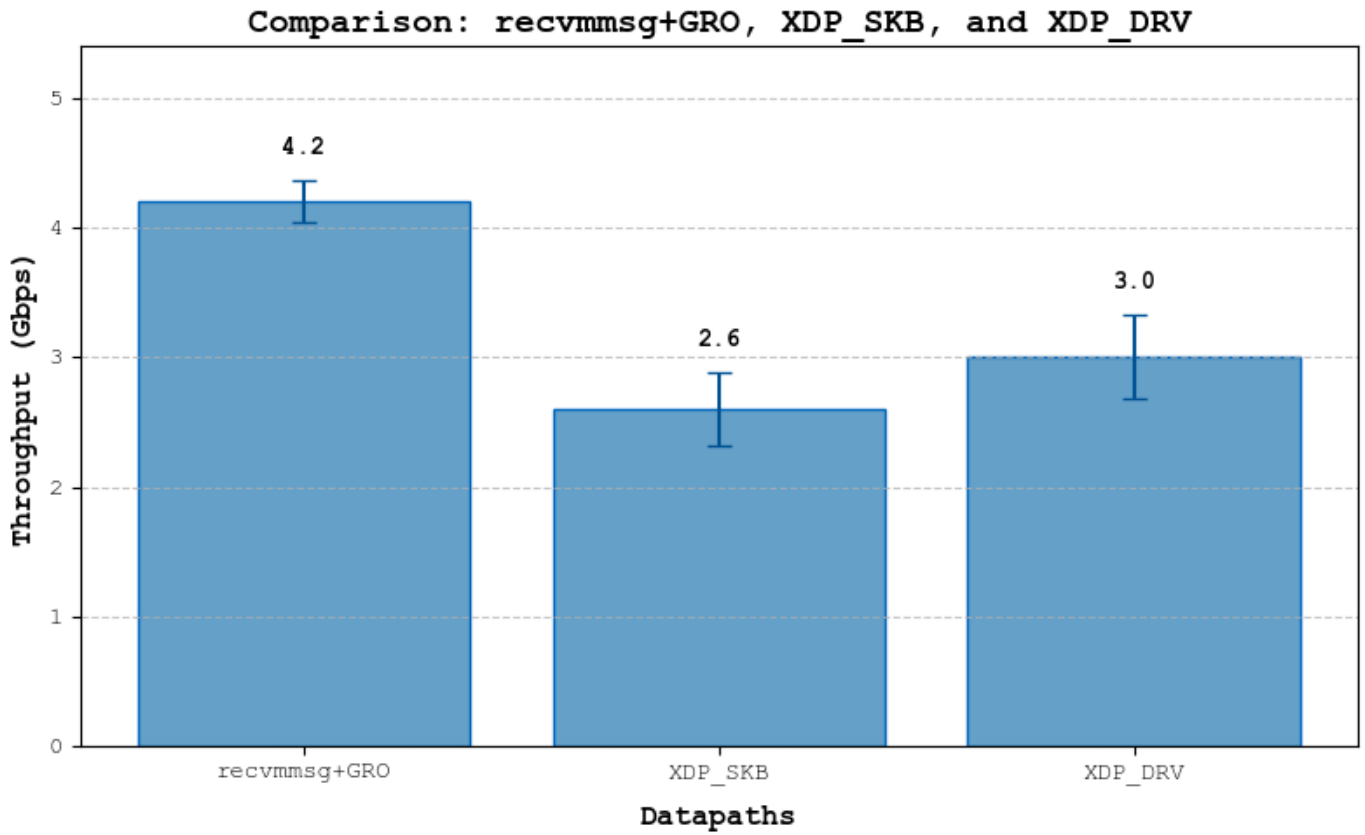
- Start with baseline MsQuic datapath (recvmmsg+GRO)
- Switch datapath to AF_XDP XDP_SKB mode



Export to pdf instead of png

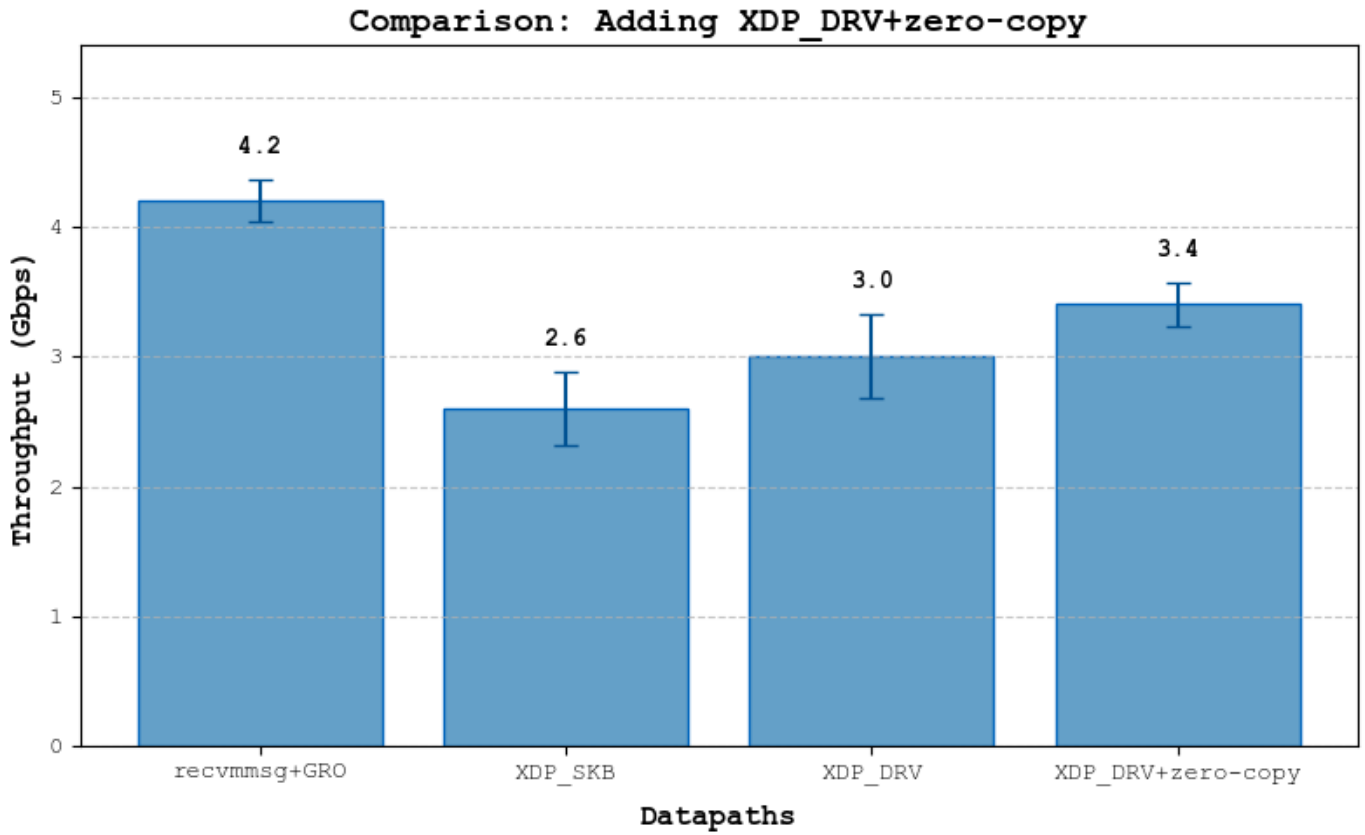
Measurements

- Move the XDP hook point closer to the driver with XDP_DRV
- No longer constructing sk_buffs, so less overhead per packet



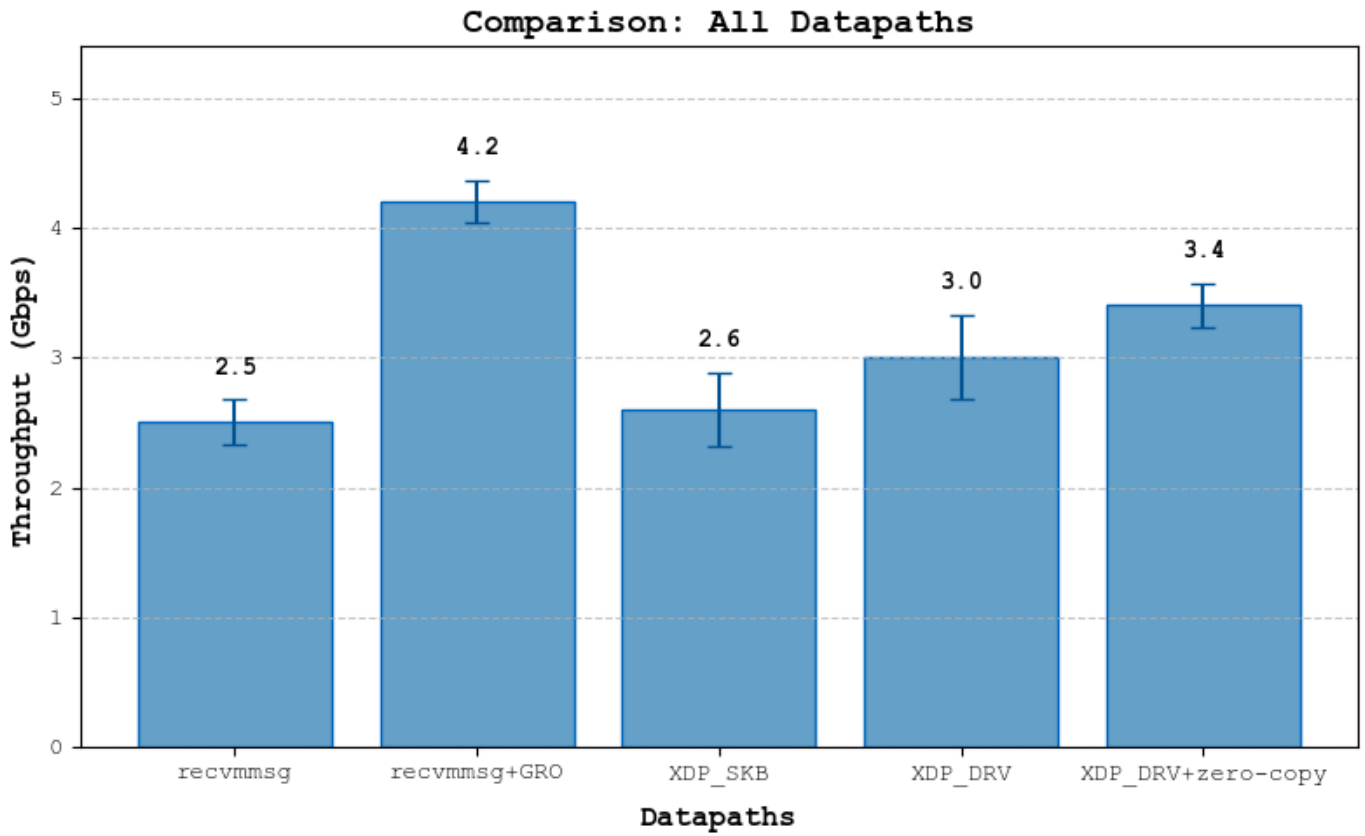
Measurements

- Eliminate an unnecessary additional copy to userspace with XDP_ZEROCOPY
- NIC now DMAs packet directly into UMEM frame



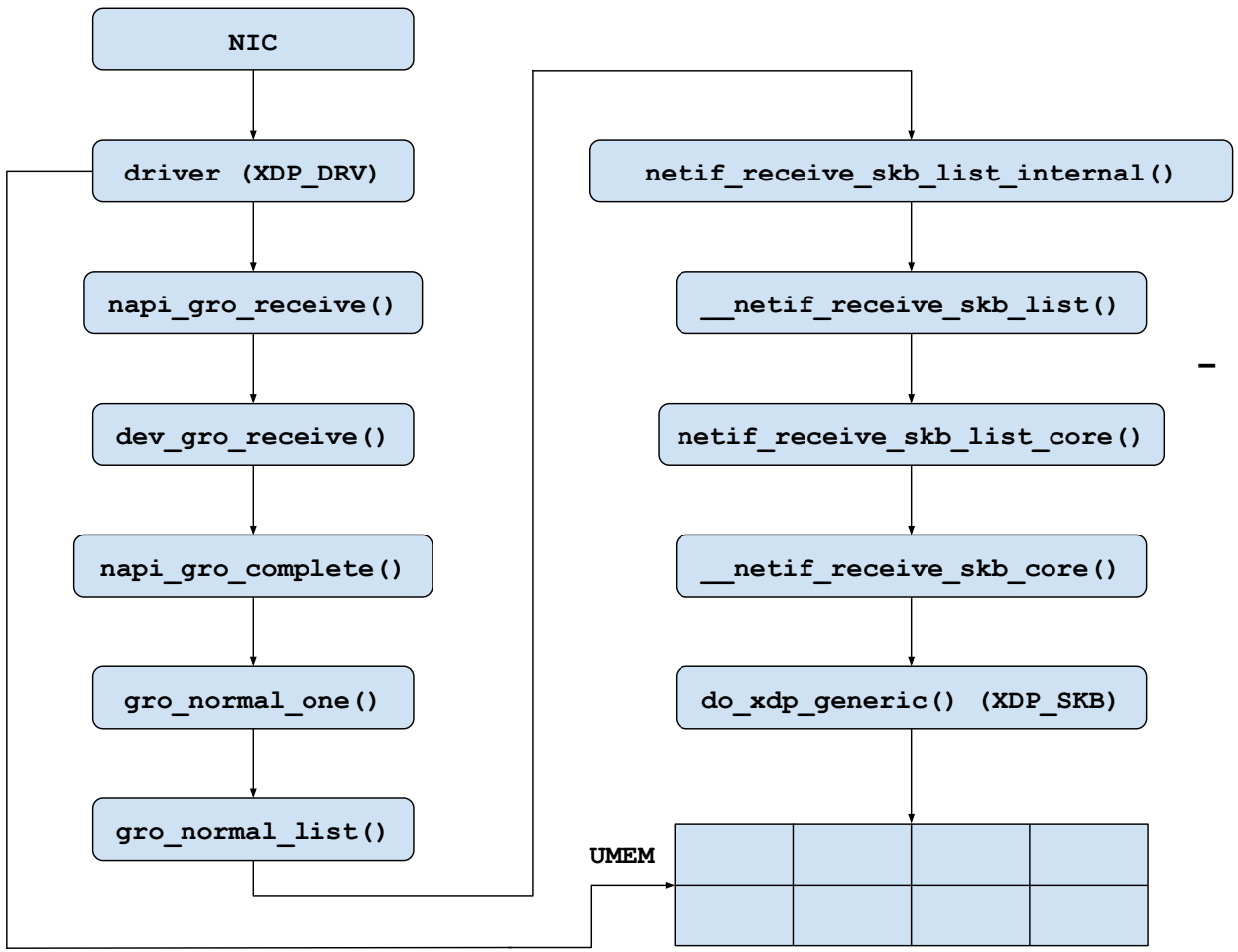
Measurements

- GRO is the main reason the default datapath is so fast
- Can we use GRO with XDP?



GRO for XDP

- Fragmentation with XDP now supported by some NIC drivers [11]
- `rx-gro-hw` with XDP_DRV or GRO with XDP_SKB when HW offload not available



Related Work

```
- Need nicer format. Cite DPDK thesis paper
- related work is to understand what is already
there.
- think of what message you want to transmit to the audience.
- AF_XDP = userspace + kernelspace (half-half?)
- lxin-quic = userspace + kernelspace (more kernelspace)
- DPDK userspace
```

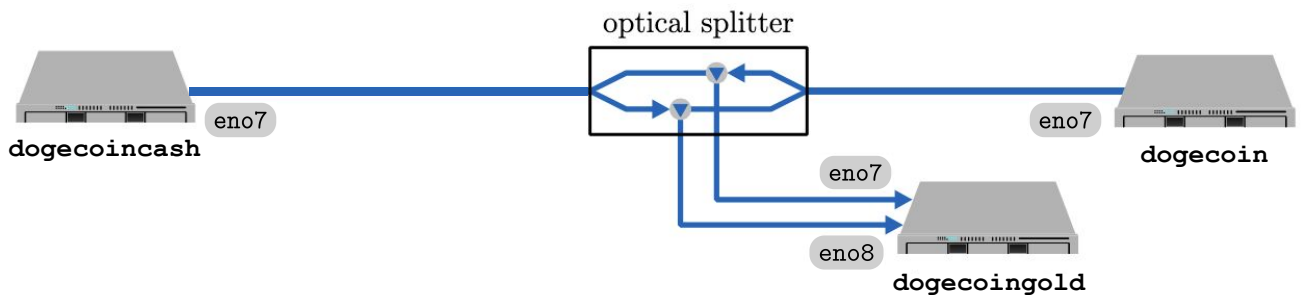
- DPDK [12] enables kernel bypass and allows application to connect directly to the NIC, reducing kernel overhead
- lxin-quic [13] brings QUIC into the kernel, reducing context switches and copying between kernel and userspace
- *QUIC Performance Improvements Using Socket Optimizations and Kernel Bypass* [14] investigates methods to improve QUIC performance by means of patching MsQuic with DPDK and using socket optimizations

```
- Compare related work to my work
- this should show why xdp approach is worthy of investigation
- quic in the kernel means you fucked if bug occurs.
- tells why you looking at xdp
```

Next Steps

- Investigate AF_XDP latency with clock-sychronized splitter and client to measure packet latency based on timing of arrived packets
- Create taxonomy for QUIC optimizations in terms of implementation complexity, performance improvement potential, unwanted side effects, flexibility, privilege level usage, and dynamic vs. static loading

- Be prepared to answer what do you still have to find out to answer this.



Conclusion

```
- Switch next steps with conclusion
```

- AF_XDP is not able to achieve a higher throughput over recvmmsg()+GRO
- XDP fragmentation support enables AF_XDP+GRO to further increase performance
- AF_XDP skipping the network stack may benefit latency
- AF_XDP represents one of many methods to reduce packet IO, each with its strengths and weaknesses

```
- XDP is way easier to integrate than DPDK
- Depends on the use case whether XDP makes
                sense or not.
```

Bibliography

[1] Mike Bishop.
HTTP/3.
RFC 9114, June 2022.

[2] Cloudflare.
Cloudflare radar year in review 2024, 2024.
Accessed: 2024-12-14.

[3] Jana Iyengar and Martin Thomson.
QUIC: A UDP-Based Multiplexed and Secure Transport.
RFC 9000, May 2021.

[4] Xumiao Zhang, Shuowei Jin, Yi He, Ahmad Hassan, Z. Morley Mao, Feng Qian, and Zhi-Li Zhang.
Quic is not quick enough over fast internet.
In *Proceedings of the ACM Web Conference 2024*, WWW '24, page 2713–2722. ACM, May 2024.

[5] Benedikt Jaeger, Johannes Zirngibl, Marcel Kempf, Kevin Ploch, and Georg Carle.
Quic on the highway: Evaluating performance on high-rate links.
pages 1–9, 06 2023.

[6] Marten Gartner.
Recapitulating af_xdp.
https://medium.com/high-performance-network-programming/recapitulating-af-xdp-ef6c1ebead8, 2022.
Figure: "The flow of packets using a normal socket and AF_XDP", retrieved from the article.

[7] Bastien Dhiver.
A story about af_xdp, network namespaces, and a cookie.
https://blog.cloudflare.com/a-story-about-af-xdp-network-namespaces-and-a-cookie/, 2022.
Cloudflare Blog.

Bibliography

[8] Microsoft.
msquic: Cross-platform, c implementation of the ietf quic protocol.
https://github.com/microsoft/msquic, 2019.
Accessed: 2025-04-17.

[9] Brendan Gregg.
Flamegraph: Stack trace visualization tool.
https://github.com/brendangregg/FlameGraph, 2016.
Accessed: 2025-04-17.

[10] Tallinn.
https://kaunas.net.cit.tum.de/topology_files/tallinn.pdf, 2024.
Accessed on 2025-04-17.

[11] Tariq Toukan.
net/mlx5e: Rx, add xdp multi-buffer support in striding rq.
https://github.com/torvalds/linux/commit/f52ac7028bec22e925c8fece4f21641eb13b4d6f, April 2023.
Commit f52ac7028bec22e925c8fece4f21641eb13b4d6f in the Linux kernel source tree.

[12] Linux Foundation.
Data plane development kit (DPDK), 2015.

[13] Xin Long.
In-kernel quic implementation with userspace handshake.
https://github.com/lxin/quic, 2023.
Accessed: 2025-04-20.

[14] Johannes Späth.
Quic performance improvements using socket optimizations and kernel bypass.
Master's thesis, Technical University of Munich, Garching, Bavaria, 2023.